

Unit 2: Python Control Flow, Functions & Modules • Topic 5

Recursion — Functions Calling Themselves, with Examples like Factorial

Understanding recursive functions, base and recursive cases, and worked examples: factorial and Fibonacci.

So far, functions have called other functions. **Recursion** is the special case where a function calls **itself** — directly or indirectly — to solve a problem by breaking it down into smaller versions of the same problem. It is an alternative to iteration (loops) for solving certain kinds of problems, and appears frequently in mathematics and computer science.

5.1 What is Recursion?

A **recursive function** is a function that calls itself, either directly or indirectly, within its own definition. Every correct recursive function must have two essential parts:

- **Base case** – the simplest version of the problem, which the function can solve directly *without* calling itself again. This is what stops the recursion.
- **Recursive case** – the function calls itself with a *smaller / simpler* version of the original problem, moving it closer to the base case with every call.

CRITICAL RULE: A recursive function with no base case (or one that is never reached) will keep calling itself forever, eventually causing a `RecursionError: maximum recursion depth exceeded`. Every recursive function **MUST** have a reachable base case.

5.2 Worked Example: Factorial

The factorial of a non-negative integer n (written $n!$) is the product of all positive integers from 1 to n , with the special definition $0! = 1$. Mathematically:

$n! = n \times (n-1)!$ (recursive case, for $n > 0$) $0! = 1$ (base case)

```
def factorial(n):
    if n == 0:           # base case
        return 1
    else:                # recursive case
        return n * factorial(n - 1)

print(factorial(5))      # 120
```

Tracing factorial(5) step by step

Every recursive call is paused, waiting for the call it made to return, before it can complete its own calculation. This creates a chain of calls that unwinds once the base case is reached:

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * (1 * factorial(0)))))
= 5 * (4 * (3 * (2 * (1 * 1)))) <- base case factorial(0) = 1 reached here
= 5 * (4 * (3 * (2 * 1))) = 5 * (4 * (3 * 2)) = 5 * (4 * 6) = 5 * 24 = 120
```

Call	n	Waiting on	Returns (once known)
1st call	5	factorial(4)	$5 \times 24 = 120$
2nd call	4	factorial(3)	$4 \times 6 = 24$
3rd call	3	factorial(2)	$3 \times 2 = 6$
4th call	2	factorial(1)	$2 \times 1 = 2$
5th call	1	factorial(0)	$1 \times 1 = 1$
6th call	0	base case — no further call	1

5.3 Worked Example: Fibonacci Sequence

The Fibonacci sequence (0, 1, 1, 2, 3, 5, 8, 13 ...) is defined as: each number is the sum of the two numbers before it, with two base cases.

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(7):
    print(fibonacci(i), end=" ")
# Output: 0 1 1 2 3 5 8
```

Notice this function makes **two** recursive calls per step (unlike factorial, which makes only one) — the number of calls grows very quickly as n increases, which is a good example of why recursion is not always the most efficient choice.

5.4 Recursion vs Iteration

Aspect	Recursion	Iteration (loops)
Mechanism	Function calls itself	Repeats a block using for/while
Memory use	Higher – each call uses stack memory	Lower – no extra call overhead
Readability	Often shorter, closer to mathematical definitions	Can be more verbose for the same problem
Risk	Stack overflow if base case is missing/unreachable	Infinite loop if condition never becomes False

Aspect	Recursion	Iteration (loops)
Good fit for	Naturally recursive problems: trees, factorial, Fibonacci, divide-and-conquer	Simple repeated tasks with clear iteration counts

Every recursive solution has an iterative equivalent (and vice versa). Python also has a default recursion depth limit (typically 1000 calls) to avoid crashing the interpreter with excessively deep recursion.

Practice Questions

A. Multiple Choice Questions

- Which of the following is essential for every correct recursive function?
(A) A loop (B) A base case (C) A lambda expression (D) A global variable
- What error occurs if a recursive function never reaches its base case?
(A) SyntaxError (B) TypeError (C) RecursionError (stack overflow) (D) IndentationError
- In the factorial recursive function, the base case is:
(A) `n == 1` (B) `n == 0` returns 1 (C) `n < 0` (D) There is no base case
- How many recursive calls does the naive `fibonacci(n)` function make per non-base call?
(A) Zero (B) One (C) Two (D) `n` calls
- Which of these problems is most naturally suited to a recursive solution?
(A) Printing numbers 1 to 100 (B) Calculating factorial of a number (C) Reading a single input value (D) Adding two numbers

B. True / False

- A recursive function without a base case will eventually cause a RecursionError.
- Every recursive solution can, in principle, be rewritten using iteration.
- Recursion generally uses less memory than an equivalent loop-based solution.
- In `factorial(5)`, the base case `factorial(0)` is the FIRST call made.

C. Short Answer Questions

- Define recursion and explain the role of the base case and recursive case with an example.
- Why must a recursive function's base case be reachable? What happens if it is not?
- Compare recursion and iteration in terms of memory usage and typical use cases.
- Trace `factorial(4)` step by step, showing each call and its return value.

D. Long Answer / Descriptive Questions

- Write a recursive Python function to calculate the sum of the first `N` natural numbers, and trace its execution for `N = 4`, showing each call and how the final result is built up.
- Explain the Fibonacci sequence and write a recursive function to generate it. Discuss why this recursive approach becomes slow for larger values of `n`, and what alternative could improve it (brief mention of iteration or memoization is sufficient).

E. Programming Problems

- Write a recursive function `power(base, exponent)` that calculates base raised to exponent without using the `**` operator.
- Write a recursive function to reverse a string (Hint: `reverse(s) = reverse(s[1:]) + s[0]`, with base case an empty string).
- Write a recursive function to find the sum of digits of a given number.
- Write both a recursive and an iterative (loop-based) Python function to calculate factorial, and compare their code.

Answer Key (A & B): MCQ: 1-B, 2-C, 3-B, 4-C, 5-B | T/F: 6-True, 7-True, 8-False, 9-False